COS 214

TIVIDY: Workflow Management System

Final Report

Team Details:

Name & Surname	**Student Number**
Kendra Maree	u25070429
Riti Botha	u25062362
Chitambala Sikazwe	u25173384
Rico Geerkens	u25015304
Gary Mull	u25071247
Heinrich Muller	u25050029
Graeme Geerkens	u24566285

GitHub Repository: https://github.com/Rico101Playz/cos-214-project

Team Contributions

Name & Surname	Student Number	GitHub account
Kendra Maree	u25070429	KenSkye111-hub
Riti Botha	u25062362	Riti-Botha
Chitambala Sikazwe	u25173384	CJM007
Rico Geerkens	u25015304	rico-geerkens, Rico101Playz
Gary Mull	u25071247	GaryMull1
Heinrich Muller	u25050029	u25050029-bot, heinrich-kizmotek
Graeme Geerkens	u24566285	graeme-geerkens

GitHub Repository: https://github.com/Rico101Playz/cos-214-project

Contribution statement

- Kendra Maree: sequence diagrams (Practical 6 and final versions), the State pattern for work items, instances and stages with its tests, and a text user interface prototype.
- Riti Botha: Chain of Responsibility (approval handlers, request and decision) with its unit tests, a first version of the Decorator pattern, and the Doxyfile and documentation comments.
- Chitambala Sikazwe: repository start and the scope and scenario draft, Participant and Organisation, the Strategy pattern for assignment and the Abstract Factory policies.
- Rico Geerkens: activity diagrams, Composite, Iterator, WorkflowInstance and the first Facade, integration fixes, Doxygen comments, and the final CMake build, README and testing guide.
- Gary Mull: Command pattern, Builder, the folder structure and first CMake build, Memento and its integration with Command, README update and pull request merges.
- Heinrich Muller: class diagrams, the web server and website, company and teams, the Proxy, the move to GoogleTest, Docker, and the final restructure into the Mediator and three facades.
- Graeme Geerkens: repository set-up and the research document, the Observer pattern, the GitHub Actions workflow and CI/CD docs, and pull request merges.

Contribution insights

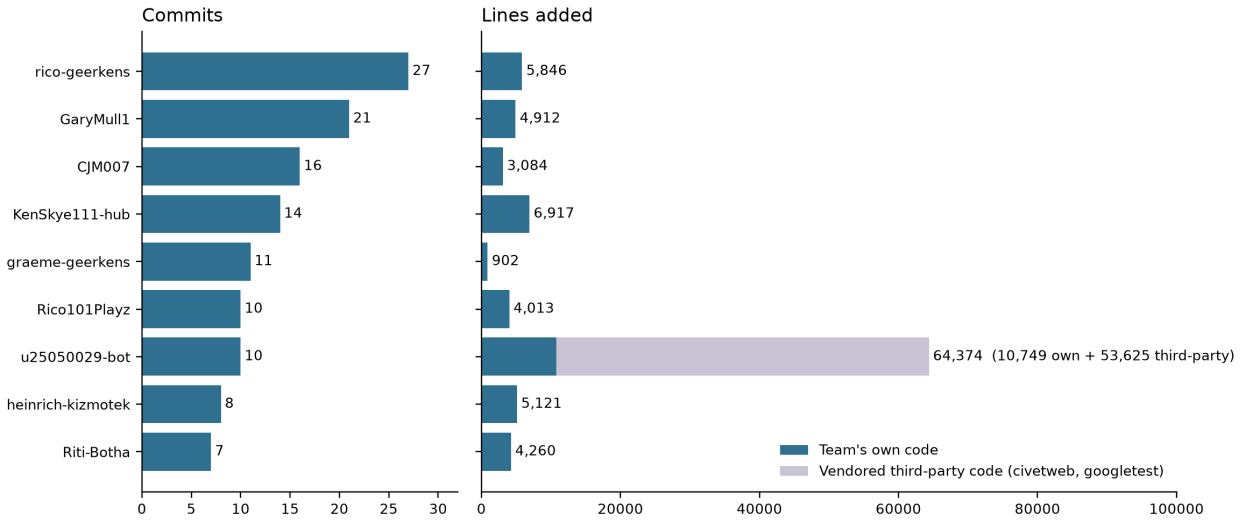


Figure 1: Commits and lines added per GitHub account

The u25050029-bot account shows 64,374 added lines. Most of this is not our own code: 53,625 of those lines are third-party libraries that were added to the repository (civetweb, the small web server, and googletest, the test framework). Only 10,749 lines are our own work. Line counts are also inflated in a few other places, for example by commits that only changed line endings, the generated Doxyfile, and the Command pull request that was reverted and applied again.

Task 1: Workflow Research

We looked at Trello and Jira (as in Practical 6) and at the Workflow Management Coalition reference model, the workflow patterns research and BPMN. Each question below ends with the decision it led to in TIVIDY.

1.1 Structure research

- A workflow is a structured sequence of work, and a workflow management system is the engine that runs it (Hollingsworth, 1995). TIVIDY is the engine; a process is supplied as a definition.
- A definition is a reusable blueprint, an instance is one run of it. Each TIVIDY instance keeps its own copy of the definition, so editing a definition never changes running work.
- Work items are separate objects that move through states (Jira issues, Trello cards). In TIVIDY each task has its own state object (State pattern).
- Nested work (Jira sub-tasks, BPMN sub-processes) is modelled as a tree of stages and tasks (Composite pattern).
- Definitions name roles, not people (Atlassian, 2020; Russell et al., 2005). People are picked when the work is assigned.

1.2 Runtime research

- Assignment is configurable: Jira uses ordered rules or round-robin (Atlassian, 2019). TIVIDY swaps assignment strategies by policy (Strategy pattern).
- Approval has an approve path and a reject path, and can have several levels (Atlassian, 2024). TIVIDY passes approval requests along a chain of handlers.
- Rules, permissions and notifications are configurable, while the core (item, state, move) stays fixed (Atlassian, 2024; van der Aalst et al., 2003).
- Events are sent out to listeners such as feeds and webhooks. TIVIDY publishes events on an event bus (Observer pattern).
- History is recorded with who did what and when. TIVIDY records every change as a command and can undo it from saved snapshots.
- The engine is reusable because it only knows items, states and transitions (OMG, 2011).

1.3 Assumptions

- Data is kept in memory and saved to a scenario text file.
- The server runs on one trusted machine (it listens on 127.0.0.1).
- One company runs per server, with teams and people loaded from the scenario.
- Logins are by name only; there is no password authentication.
- Escalation is started by a person; due hours are shown but not enforced by a timer.

1.4 TIVIDY boundaries

In scope	**Out of scope**
Definitions, instances, stages and tasks	The business work itself
Task lifecycle, assignment, approvals, escalation	Real authentication and accounts
Teams, roles and permissions	A database
Events, activity log, undo and redo	External systems (Git, CI, ERP)
A multi-user website with live updates	Version numbers on definitions

Task 2: Industries

TIVIDY runs both processes without changes to the engine. Each process is only a definition (stages, tasks, roles and a policy).

2.1 Software engineering: Feature Delivery

Process: Intake (technical spec), Build (back-end and front-end work as sub-stages), Verify (integration tests and code review), Release (deploy).

Role	**Responsibility**
Owner	Signs off the finished project
Manager	Approves the spec and the code review, reassigns work
Tech Lead	Writes the spec, approves reviews
Reviewer	Reviews code
Developer, Designer	Build the back end, front end and screens
QA	Runs the integration tests
Release Manager	Approves the release

Branching and decisions:

- An approver approves or rejects each task. A rejected task goes back to the worker.
- The policy decides how strict the process is: SmallTeam, BusyTeam or HighRisk (HighRisk runs stages one after the other and adds Tech Lead and Release Manager approval).
- Blocked tasks wait: the UI task waits for the screens and the API, the code review waits for the tests.
- At the end the Owner signs off or flags issues, which reopens the project with a fixes stage.

Failure path	**How TIVIDY handles it**
Review or test fails	Task is rejected and reworked
Approver does not respond	Escalated by hand up the chain
Wrong action taken	Undo or redo
Feature cancelled	Project cancelled, all open tasks cancelled
Problem found after delivery	Issue flagged at sign-off, fixes stage added

Outcomes: Completed (signed off), AwaitingSignOff, or Cancelled with a reason.

2.2 Manufacturing: Production Order

Our example is a small machining company ("Karoo Precision Components") making batches of aluminium parts. The stages follow normal shop-floor control and ISO 9001 quality checks (Groover, 2015; ISO, 2015): Order release (route, kit material, prove the CNC program), First article (set up, first piece, FAI report), Production run (machining then assembly), Final inspection (inspect, certificate of conformity) and Pack and ship.

Role	**Responsibility**
Plant manager (Owner)	Closes the order, approves concessions
Production and quality managers	Approve routing and the certificate, chair the review board

Role	Responsibility
Planner, Storeman	Release orders, issue material, pack and ship
Machinist, Assembler	Set up, machine and assemble
Quality inspector	Inspects first piece, batch and shipment
Quality engineer	Approves the FAI and inspection results

Branching and decisions:

- First piece conforms? If not, the set-up is rejected and cut again.
- Batch conforms? If not, a nonconformance report is added as a sub-task and the review board decides: rework, use as is, or scrap (ISO, 2015).
- Machining cannot start before the first article stage is done, and nothing ships before the certificate is approved.

Failure path	How TIVIDY handles it
Missing material certificate	Kit task rejected with the reason
Machine breakdown or absent machinist	Task reassigned to the least-loaded machinist
Nonconforming parts	Sub-task holds the task until the review board approves
Whole batch scrapped	Order cancelled with the reason
Customer complaint	Issue flagged at sign-off

Outcomes: Completed (shipped and closed), or Cancelled with a reason. We wrote this process as a scenario file and ran it in the console app under the BusyTeam and HighRisk policies.

Task 3: Updated Requirements

ID	Requirement	Status
R1	Store workflow definitions	Met (no version numbers)
R2	Create independent instances from a definition	Met
R3	Stages, tasks and sub-tasks, sequential or parallel	Met
R4	Task lifecycle with nine states, invalid actions refused	Met
R5	Assignment by role, least load or rule	Met
R6	Approvals, rejection and escalation	Met (escalation by hand)
R7	Events that other parts can react to	Met
R8	External system integration	Dropped
R9	History, undo and redo	Met
R10	Progress and workload reporting	Partly (progress and activity log)
R11	Permissions per role	New, met (Proxy)
R12	Several users at once with live updates	New, met (web server)
R13	Company, teams and people	New, met
R14	Project sign-off, blockers, priorities	New, met

Task 4: UML Diagrams

4.1 Activity diagrams

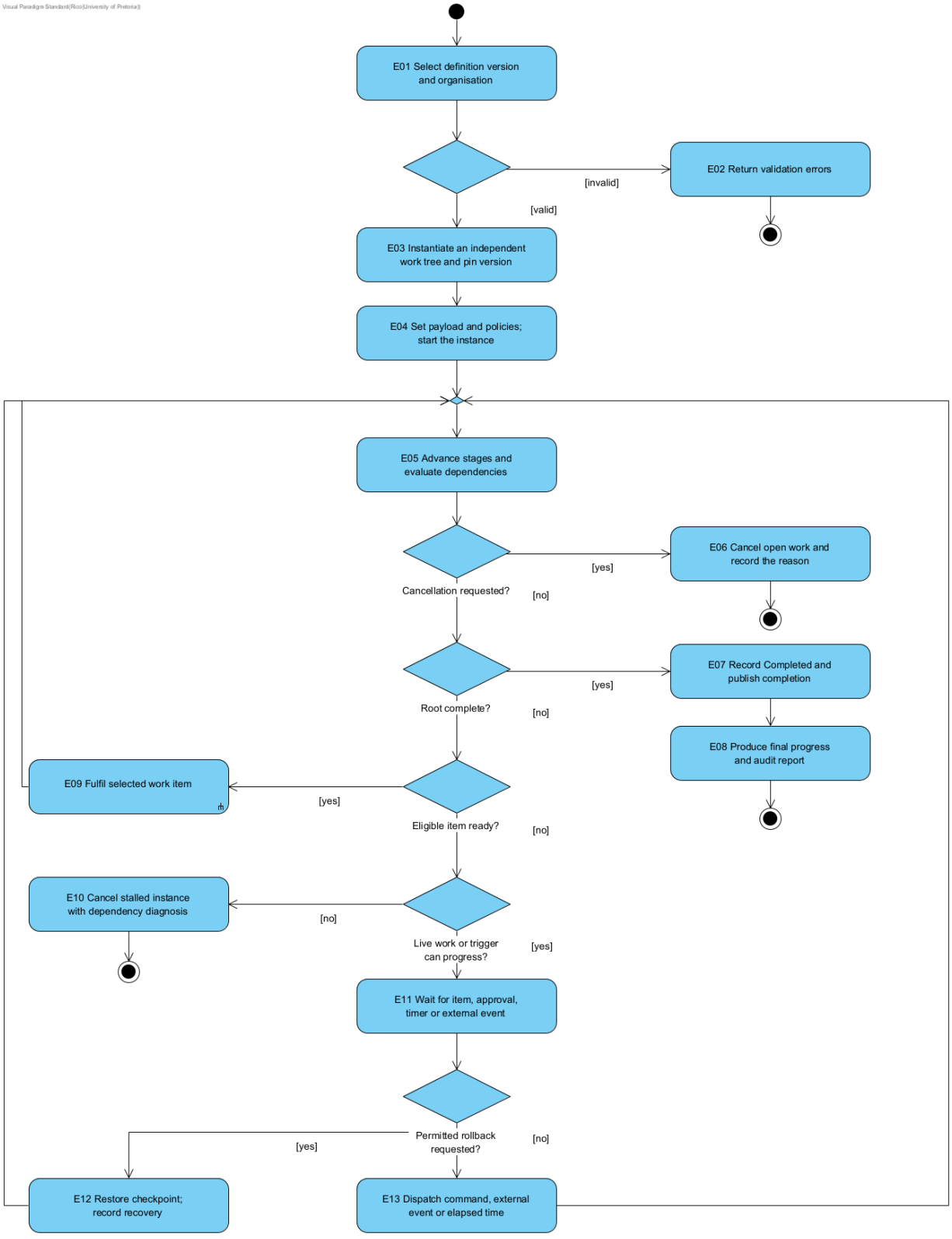


Figure 2: Generic execution

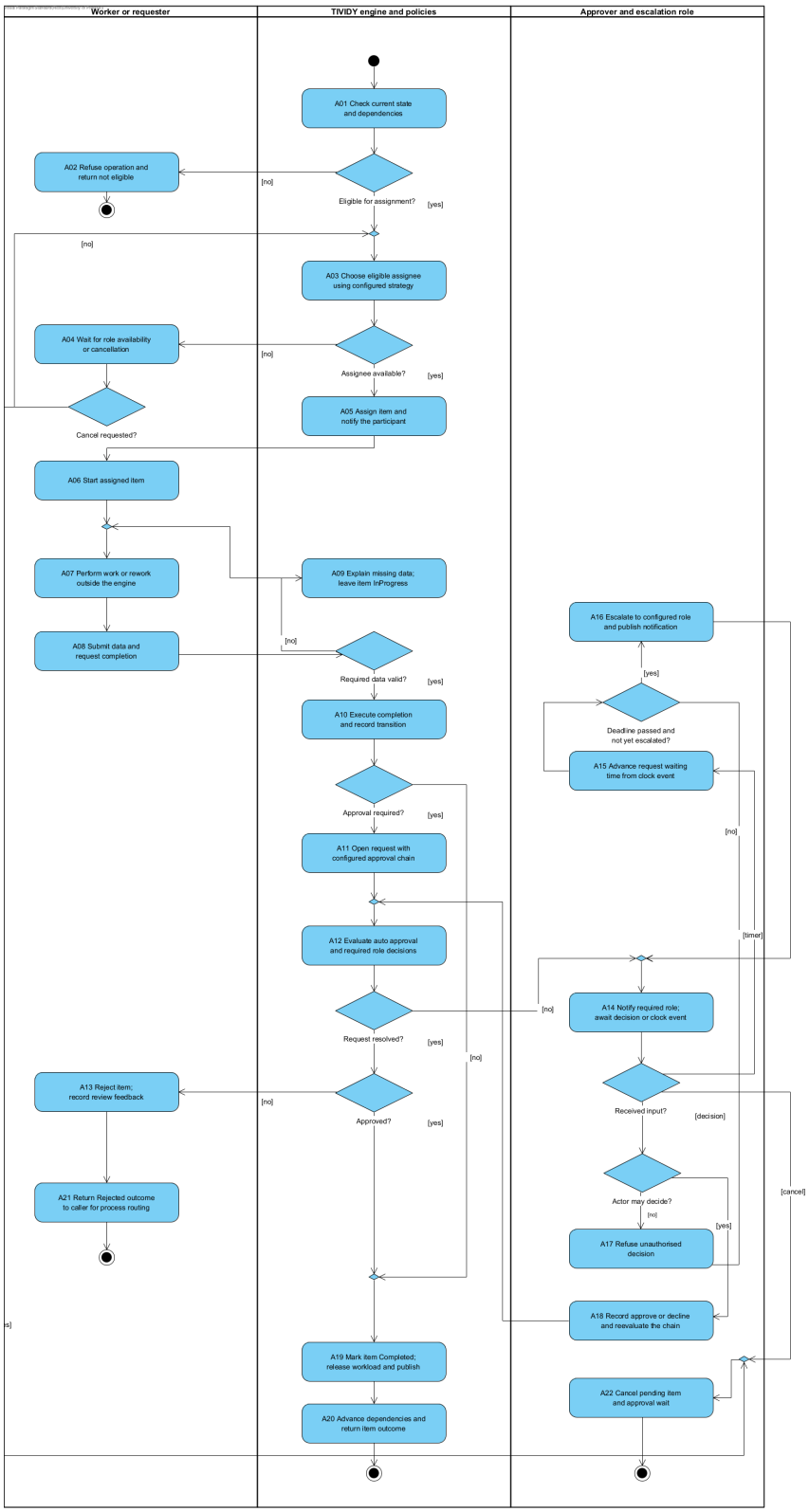


Figure 3: Assignment and approval

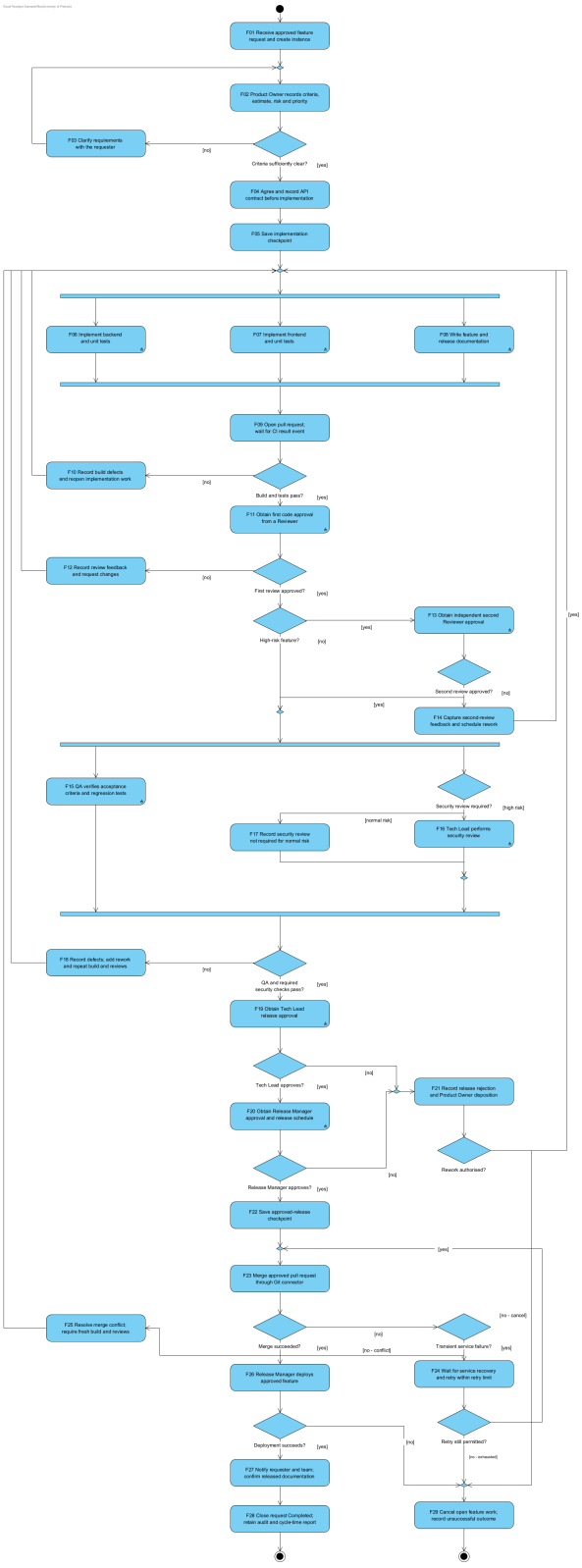


Figure 4: Software feature delivery

Changes from the implementation: there is no version selection, no timer events and no Git or CI connector. Escalation and cancelling a stalled project are done by a person. A project that needs sign-off waits in AwaitingSignOff before it completes.

4.2 Class diagram

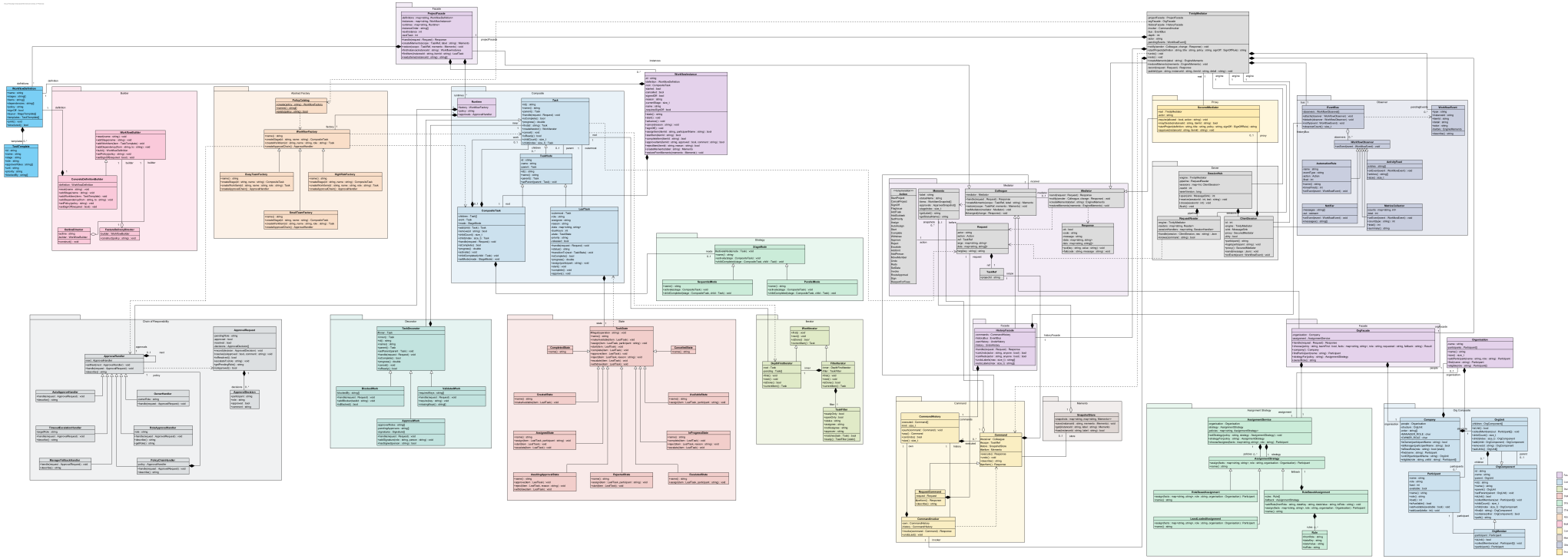


Figure 5: Final class diagram

Practical 6	Final design
Tividy facade	TividyMediator with ProjectFacade, OrgFacade and HistoryFacade
WorkNode, CompositeStage, LeafWorkItem	Task, TaskNode, CompositeTask, LeafTask
WorkItemState	TaskState
WorkItemDecorator, PriorityWork, AuditedWork	TaskDecorator, BlockedWork, ValidatedWork, ApprovalWork
ReadyOnlyIterator	FilterIterator with TaskFilter
WorkCommand and one class per action	Command and RequestCommand
InstanceMemento	Memento and engine snapshots
Startup, Onboarding factories	SmallTeam, BusyTeam, HighRisk factories
FeatureDeliveryProcess	FeatureDeliveryDirector, OutlineDirector
Visitor, Adapter	Dropped
(none)	Mediator, Proxy, Server package

4.3 Sequence diagrams

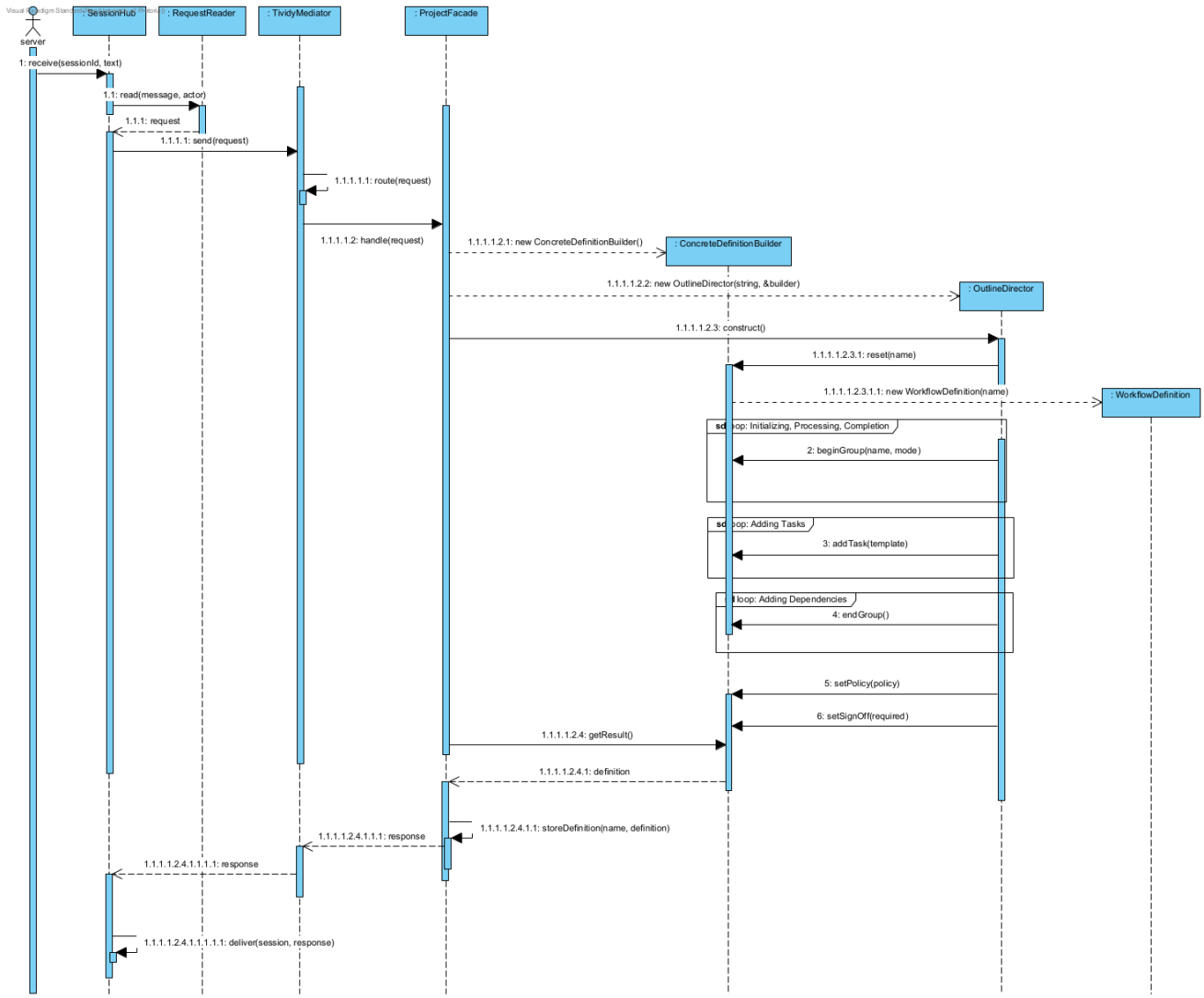


Figure 6: Creating a workflow definition

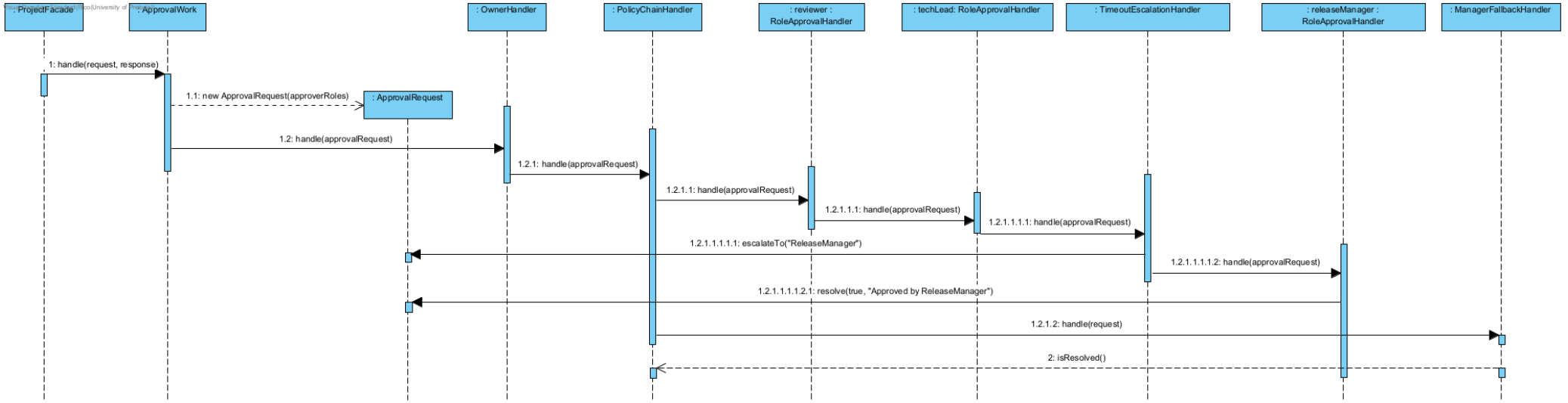


Figure 7: Escalation up the approval chain

Task 5: Design Patterns

Pattern	Classes	Why we use it
Facade	ProjectFacade, OrgFacade, HistoryFacade	Each hides one subsystem (projects, company, history) behind a few calls
Mediator	Mediator, TividyMediator, Colleague	All requests go through one object, so the facades never call each other
Proxy	SecuredMediator	Checks the user's role before a request reaches the engine
Composite	Task, CompositeTask, LeafTask; OrgComponent, OrgUnit	Stages, tasks and sub-tasks (and teams and people) are treated the same way
Decorator	TaskDecorator, BlockedWork, ValidatedWork, ApprovalWork	Adds blockers, data checks and approvals to a task without new task classes
State	TaskState and nine concrete states	Each state allows only its own actions, no large if/else
Iterator	WorkIterator, DepthFirstIterator, FilterIterator	Walks the task tree without exposing it, for example to list ready work
Strategy	StageMode (Sequential, Parallel); AssignmentStrategy (role, least loaded, rule)	Swaps how stages run and how people are picked per policy
Chain of Responsibility	ApprovalHandler and its handlers	Approval levels and escalation can be added or reordered
Abstract Factory	WorkflowFactory, SmallTeam, BusyTeam, HighRisk factories	Each policy builds a matching set of stages, tasks and approval chain
Builder	WorkflowBuilder, ConcreteDefinitionBuilder, directors	Builds a definition step by step
Command	Command, RequestCommand, CommandInvoker	Every change is an object, so it can be recorded and undone
Memento	Memento, SnapshotStore	Saves state for undo, redo and restore without exposing it
Observer	EventBus, WorkflowObserver	The website, activity log and other listeners react to events

Changes since Practical 6: Mediator was added because the single Tividy facade grew too large, Proxy was added for permissions when the website got several users, Visitor was dropped because the iterators already cover reporting, and the Adapter was dropped with the external integration.

Task 6: Implementation

A browser sends a message over a WebSocket. The SessionHub gives it to the RequestReader, which turns it into a Request. The user's SecuredMediator checks the permission and passes it to the TividyMediator. The mediator runs it as a command on the right facade, saves a snapshot so it can be undone, and publishes events. Every open browser tab is an observer, so all users see the change straight away. The console app uses the same engine.

6.1 Design decision history

Date	Decision
1-7 Oct	Class diagram, scope, activity and sequence diagrams (Practical 6)
19-25 Oct	Each pattern built on its own feature branch and merged by pull request
20-27 Oct	Web server, teams, approvals board and live updates added
27 Oct	All tests moved to GoogleTest
30 Oct-1 Nov	Code restructured into the final design: Mediator, three facades, Proxy, one folder per pattern
1-2 Nov	CI/CD workflow, Doxygen comments, README, final class diagram

6.2 Requirements to classes, patterns and tests

Req.	Classes	Pattern	Tests
R1, R2	WorkflowDefinition, ProjectFacade, WorkflowInstance	Builder, Facade	Builder, WorkflowInstance
R3	CompositeTask, LeafTask, StageMode	Composite, Strategy	CompositeTree, StageMode
R4	TaskState and states	State	TaskState, InstanceState
R5	AssignmentStrategy and subclasses	Strategy	Strategy
R6	ApprovalHandler chain	Chain of Responsibility	ChainOfResponsibility, ApprovalFlowTest
R7	EventBus, WorkflowObserver	Observer	Observer
R9	Command, HistoryFacade, Memento	Command, Memento	Command, Memento
R11	SecuredMediator	Proxy	Proxy
R12	SessionHub, ClientSession	Observer, Mediator	Server
R13	Company, OrgUnit, Participant	Composite	CompanyComposite
R14	BlockedWork, ApprovalWork	Decorator	DecoratorTest, StructuredWorkflow

Task 7: Build, Run and Test

```
mkdir build && cd build
cmake ..
cmake --build .
./TIVIDY_Server --ports 8080
./TIVIDY_App
ctest
```

TIVIDY_Server serves the website on http://localhost:8080/ and asks which scenario in scenarios/ to open. TIVIDY_App is the console version. DOCKER.md explains how to run everything with Docker instead.

7.1 Test results

The GoogleTest suite has 123 tests in 25 suites, one file per pattern. We built main on 2 November 2026 and all 123 tests passed (548 ms).

7.2 Git and engineering evidence

- Branches: main, dev, demo and one feature branch per pattern or task.
- Work was merged through 31 pull requests, from the Practical 6 diagrams (#1-#4) to the final merge of dev into main (#31).
- Commit messages follow the conventional style (feat, fix, test, docs, refactor).
- GitHub Actions builds the project and runs the tests on every push and pull request to dev and main, and keeps the test report.
- A Dockerfile and docker-compose file run the server in a container.

References

Atlassian (2019) Using automation for round-robin assignee which is different than reporter. Available at: https://community.atlassian.com/forums/discussion/1202858/using-automation-for-round-robin-assignee-which-is-different-than-the-reporter (Accessed: 5 October 2026).

Atlassian (2020) What are permission schemes in Jira? Available at: https://support.atlassian.com/jira-cloud-administration/docs/what-are-permission-schemes-in-jira/ (Accessed: 3 October 2026).

Atlassian (2024) Introduction to Jira workflows. Available at: https://www.atlassian.com/software/jira/guides/workflows/overview (Accessed: 3 October 2026).

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley.

Groover, M.P. (2015) Automation, Production Systems, and Computer-Integrated Manufacturing. 4th edn. Boston, MA: Pearson.

Hollingsworth, D. (1995) The Workflow Reference Model. Document TC00-1003, Issue 1.1. Workflow Management Coalition.

International Organization for Standardization (2015) ISO 9001:2015 Quality management systems - Requirements. Geneva: ISO.

Object Management Group (2011) Business Process Model and Notation (BPMN), Version 2.0. Needham, MA: OMG.

Russell, N., van der Aalst, W.M.P., ter Hofstede, A.H.M. and Edmond, D. (2005) 'Workflow resource patterns', in Advanced Information Systems Engineering: CAiSE 2005. Berlin: Springer, pp. 216-232.

van der Aalst, W.M.P., ter Hofstede, A.H.M., Kiepuszewski, B. and Barros, A.P. (2003) 'Workflow patterns', Distributed and Parallel Databases, 14(1), pp. 5-51.